ScoreDrop Unity Package - User Manual

Version 1.0.0
March 2026
Official Unity Integration for ScoreDrop API

Table of Contents

1. Introduction
2. Installation
3. Quick Start
4. Component Reference
    - ScoreDropManager
    - ScoreDrop_Game
5. Prefabs
6. API Integration
7. Troubleshooting
8. FAQ
9. Support

---

1. Introduction

ScoreDrop is a simple, modern leaderboard API designed for indie game developers. This Unity package provides a complete, ready-to-use integration with the ScoreDrop service.

Features

- **Ready-to-use UI** - Prefab with leaderboard, name editing, and score simulation
- **Automatic player identification** - Each player gets a unique `player_id` stored in PlayerPrefs
- **Live leaderboard** - Paginated display with current player highlighting
- **Name editing** - Players can change their display name
- **Score simulation** - Built-in +10 button for testing (replace with your game logic)
- **Persistent data** - Player name and best score saved locally
- **Multi-platform** - Works on all platforms Unity supports

---

2. Installation

2.1 Via Unity Package Manager (UPM)

1. Open Unity and go to `Window > Package Manager`
2. Click the `+` button and select **"Add package from git URL"**

3. Enter: `https://github.com/Guardabarrancoestudio/scoredrop-unity.git`
4. Click **Add**

2.2 Manual Installation

1. Download the latest `.unitypackage` from the Releases page
2. In Unity, go to `Assets > Import Package > Custom Package`
3. Select the downloaded file and click **Open**
4. Click **Import** in the Import dialog

2.3 Dependencies

This package requires **TextMeshPro**. If not already installed, Unity will prompt you to import it automatically.

---

3. Quick Start

3.1 Step 1: Add the Manager

Option A: Drag the `ScoreDropManager` prefab into your first scene.
Option B: Add the `ScoreDropManager` component to any existing GameObject.

3.2 Step 2: Configure API Keys

In the Inspector, set your:

- **API Key** (from your leaderboard)
- **Leaderboard ID** (UUID from your leaderboard)

API Key Configuration

3.3 Step 3: Add the UI

1. Drag the `ScoreDropCanvas` prefab into your scene
2. In the Inspector, assign the references:
    - `Name HUD`: Text that shows player name
    - `Score HUD`: Text that shows current score
    - `Player Name Input`: Input field for editing
    - `Leaderboard Container`: Where scores will appear
    - `Score Entry Prefab`: Your leaderboard entry prefab
    - All buttons (Submit, Refresh, Next, Prev, Edit Name, Add Score)

3.4 Step 4: Run the Demo

- Open the `DemoScene` to see a working example
- Press **Play** and test:

- Click **+10** to add points locally
- Click **Submit** to send your score to the API
- Click **Edit Name** to change your display name
- Use **Next/Prev** to navigate pages

---

4. Component Reference

4.1 ScoreDropManager

The core singleton component that handles all API communication.

Public Methods

Method	Description
`GeneratePlayerId()`	Returns a new unique GUID for player identification
`AddScore(string playerName, int score, string playerId, Action<AddScoreResponse> onSuccess, Action<string> onError)`	Sends a score to the leaderboard
`GetLeaderboard(int limit, int page, Action<LeaderboardResponse> onSuccess, Action<string> onError)`	Retrieves leaderboard data

Inspector Properties

Property	Description
`API Key`	Your ScoreDrop API key (from leaderboard)
`Leaderboard ID`	Your leaderboard's unique identifier

`Base URL`	API endpoint (default: `https://leaderboard-game.vercel.app/api`)

4.2 ScoreDrop_Game

The main UI controller that manages player interaction.

Inspector Properties

Category	Property	Description
Player HUD	`Name HUD`	Text showing player name (always visible)
	`Score HUD`	Text showing current score (always visible)
Input Fields	`Player Name Input`	Input field for name editing
Score Button	`Add Score Button`	Button to add +10 locally
Buttons	`Submit Button`	Sends current score to API
	`Refresh Button`	Reloads leaderboard
	`Next Page Button`	Next page of leaderboard
	`Prev Page Button`	Previous page
	`Edit Name Button`	Enables name editing mode
Settings	`Scores Per Page`	Number of entries per page

UI Container	`Name Input Container`	GameObject containing name input (hidden when not editing)
Default Values	`Default Score`	Courtesy points for new players (default: 100)

---

5. Prefabs

5.1 ScoreDropManager Prefab

A singleton GameObject that should exist in your first scene. It will persist across scenes using `DontDestroyOnLoad`.

5.2 ScoreDropCanvas Prefab

Complete UI layout containing:

- Player HUD (name and score)
- Name input field (hidden by default)
- All control buttons
- Leaderboard container
- Status and feedback texts

5.3 Score_Entry_Prefab

Template for individual leaderboard entries. Must contain 3 TextMeshPro texts for:

- Rank
- Player Name
- Score

---

6. API Integration

6.1 How Scoring Works

1. **Local Score**: Players accumulate points locally (simulate gameplay)
2. **Submit**: When ready, click Submit to send to API
3. **API Decision**: The API only updates if the score is higher than previous best
4. **Feedback**: Clear messages for each outcome

6.2 Player Identification

- Each player gets a unique `player_id` (GUID) on first launch

- This ID is stored in PlayerPrefs and persists between sessions
- All scores are tied to this ID, not the player name

6.3 Name Editing

- Players can change their display name at any time
- New name is saved locally and sent to API on next submit
- The API updates the name while preserving the player's score history

6.4 API Response Messages

Message	Meaning
`"Score not updated (existing score is higher)"`	Score didn't improve
`"Score updated (new score is higher)"`	New personal best
`"Score added (replaced lowest score in top)"`	Entered leaderboard, replaced lowest
`"Score added successfully"`	First score submission
`"Name updated (score unchanged)"`	Only name changed

---

7. Troubleshooting

7.1 "Leaderboard full" message

Problem: You see "Leaderboard full. Your score isn't high enough to enter the top."

Solution: Your score needs to be higher than the current lowest score in the top. Keep playing to improve your score.

7.2 Name not saving

Problem: Edited name doesn't appear after submit.

Solution: Make sure you click **Submit** after editing. Name changes are only saved when you submit a score.

7.3 Player ID lost

Problem: Player appears as a new player with different ID.

Solution: PlayerPrefs may have been cleared. This is normal on first launch or after clearing app data.

7.4 Connection errors

Problem: “Error: Connection failed” messages.

Solution: Check your internet connection and verify that the API URL is correct in ScoreDropManager.

---

8. FAQ

Q: Do I need to generate player_id manually?

A: No! The package automatically generates and stores a unique ID for each player.

Q: Can players change their name?

A: Yes! Click **Edit Name**, change the name, and click **Submit** to save.

Q: What happens if I submit a lower score?

A: The API ignores it and shows “Score not improved” feedback. Your best score remains unchanged.

Q: Is my data safe?

A: Yes. Player data is stored locally in PlayerPrefs. API communication is over HTTPS.

Q: Can I customize the UI?

A: Absolutely! The prefabs are fully editable. Just make sure to keep the required references.

Q: Does it work on mobile?

A: Yes! ScoreDrop works on all platforms Unity supports (PC, Mac, Linux, iOS, Android, WebGL).

---

9. Support

Documentation

- ScoreDrop API Docs
- Features & Plans
- FAQ

Contact

- **Email**: Guardabarrancoestudioapp@gmail.com
- **Website**: ScoreDrop
- **Website**: Guardabarranco
- **GitHub**: github.com/Guardabarrancoestudio/scoredrop-unity

Contributing

Found a bug? Have a feature request? Open an issue on GitHub or submit a pull request. We welcome community contributions!

© 2026 ScoreDrop. All rights reserved.

Made with for indie game developers